Robust Aggregation の失敗モードに関する定量分析計画書

3CDA2216 笠原秀星

2026 年 8 月 24 日

1 目的

本文書は，`sotsuron_plan.tex` で報告した実験結果（Adaptive Model Poisoning Attack に対する各 Robust Aggregation 手法の耐性評価）のうち，特に挙動の異なる 4 つの現象について，その要因を定量的に分析するための計画をまとめる。

Fang ら [1] の Adaptive Model Poisoning Attack は，Robust Aggregation 手法が防御として不十分であることを攻撃成功率の観点から示したが，「なぜ・どの条件で防御が破綻するのか」というメカニズムには踏み込んでいない。本研究では，既存実験で観測された以下の現象を手がかりに，この問いに定量的な説明を与えることを目指す。

2 観測された現象

- Krum / Bulyan は，Malicious Ratio 0.0（悪性クライアント不在）の場合でもラウンドごとの精度変動が大きい。
- FedAvg は，Malicious Ratio 0.2 以上で Accuracy・Loss がラウンドごとに大きく振動する。
- Trimmed Mean を Non-IID 環境で用いた場合，Malicious Ratio 0.3 のときのみ Seed 間で最終 Accuracy が 0.74–0.84 と大きくばらつく（他の Malicious Ratio では Seed 間差は 0.01 未満）。
- Bulyan は頑健性条件 $n \geq 4f + 3$ を満たさない Malicious Ratio 0.3（$100 < 123$）において，原理的に動作しない。

4 点目は集約手法の数式上の制約から自明に説明できるため考察のみで扱い，1–3 点目について新たな定量指標を導入して分析する。

3 提案する分析指標

3.1 honest クラスタの分散（benign spread）

各ラウンドにおいて，honest なクライアント更新の重心 $\bar{v}_{\text{benign}}$ からの平均距離を次式で定義する。

$$\text{spread} = \frac{1}{|B|} \sum_{i \in B} \|v_i - \bar{v}_{\text{benign}}\|_2 \tag{1}$$

ここで B は honest クライアントの添字集合，v_i はクライアント i の更新ベクトルである。IID 環境より Non-IID 環境の方がこの値が大きくなることが予想される。

3.2 malicious の押し出し距離と push ratio

攻撃適用後の malicious 更新ベクトルについても同様に，honest 重心からの平均距離を求める。

$$\text{push} = \frac{1}{|M|} \sum_{i \in M} \|v_i - \bar{v}_{\text{benign}}\|_2 \tag{2}$$

両者の比

$$\text{push_ratio} = \frac{\text{push}}{\text{spread}} \tag{3}$$

を定義する。push_ratio が小さいほど，malicious 更新は honest クラスタの分布に紛れやすく，検出・排除されにくいと解釈できる。「Non-IID 環境では honest 同士の分散が元々大きいため push_ratio が小さくなり，攻撃が識別されにくい」という仮説をこの指標で検証する。

3.3 Krum 選択結果

Krum および Bulyan の内部選択において，選ばれたクライアントが malicious か honest かを 2 値 `krum_selected_malicious` として各ラウンド記録する。Malicious Ratio 0.0 でも生じる精度変動が，honest クライアント間のどのような差異に起因するかをこの記録と spread の推移から考察する。

3.4 Trimmed Mean における生存率（今後の課題）

Trimmed Mean は座標ごとに上位・下位 β 個の値を除外してから平均を取るため，malicious 更新が座標ごとに除外対象となるか否かで攻撃の効き方が変わる。座標ごとの順位を用いて，malicious 更新が除外されずに生き残った座標の割合 `frac_dims_survived` を定義する。この指標は次元数が大きく計算コストが増えるため，まずは 3.1–3.3 節の指標を優先し，実装余力があれば追加する。

4 実装方針

上記指標は，pytorchexample/robust_strategy.py の aggregate_train 内，Adaptive Model Poisoning Attack 適用後・集約前の時点で計算する。この時点で全クライアントの更新ベクトル updates，malicious 添字 malicious_pos，集約前のグローバルモデル self._last_global_vector がすでにメモリ上に存在するため，モデル全体のベクトルを新たに保存する必要はなく，ラウンドごとの要約統計量のみを CSV へ追記すればよい。

5 実験対象条件

新指標は既存ログ（result/, logs/）には含まれないため，再実験が必要である。全条件× Ratio 細分化× Seed 数増加を単純に掛け合わせると実行数が膨大になるため（後述の見積りで 350〜400 回，計算時間にして半日以上），**探索 (Phase 1)** と**検証 (Phase 2)** の 2 段階に分けて依存関係を切ることで，網羅性を保ちながら実行数を抑える。

5.1 Phase 1：ベースグリッド＋ Ratio 細分化（探索）

まず Seed 数は 3 のまま，全手法・全条件を対象に Ratio 方向の解像度を上げて実行する。目的は「どの手法・どのデータ分布・どの Ratio 付近で不安定になるか」を広く安く特定することであり，この段階では Seed 間分布の精緻な推定は求めない。

- 集約手法：FedAvg, Median, Trimmed Mean, Krum, Bulyan の 5 手法全て
- データ分布：IID・Non-IID
- Malicious Ratio：基本は 0.0，0.1，0.2，0.3。加えて
  - Trimmed Mean（Non-IID）：0.225，0.25，0.275 を追加
  - Krum・Bulyan：0.05 刻みで 0.0 から実行可能上限（Bulyan は $n \geq 4f + 3$ を満たす範囲）まで追加
  - FedAvg：0.125，0.15，0.175 を追加
- Seed：3（全条件共通，据え置き）

概算実行数は FedAvg 42, Median 24, Trimmed Mean 33，Krum 42，Bulyan 30 の計約 171 回（1 回平均 2 分として約 5.7 時間）。

5.2 Phase 2：Seed 数増加（検証）

Phase 1 の結果，Seed 間で最終 Accuracy のばらつきが大きい（既存結果の Trimmed Mean Non-IID Ratio 0.3 のような）ことが確認された Ratio 点**のみ**を対象に，Seed を 3 から 10 へ増やして追加実行する（7 Seed 分を追加）。全 Ratio ×全手法に一律で Seed を増やすことはしない。

現時点で Seed 間ばらつきが疑われるのは Krum・Bulyan（元々 Ratio 0.0 でも変動が大きい）と Trimmed Mean（Non-IID, Ratio 0.2–0.3 付近）だが，最終的な対象点は Phase 1 の結果を見てから確定する。仮に Krum・Bulyan の全 Ratio × IID/Non-IID と Trimmed Mean Non-IID の数点が対象になった場合，概算で追加 70 回程度（約 2.3 時間）。

Phase 1 と 2 を合わせても総実行数は約 240 回・半日弱に収まり，全条件に 10 Seed ×細分化 Ratio を単純に掛け合わせる場合（350〜400 回）よりも大幅に抑えられる。

6 今後の流れ

1. robust_strategy.py に spread / push / push_ratio / krum_selected_malicious のロギングを追加する。
2. 5 節の Phase 1（ベースグリッド＋細分化 Ratio, Seed 3）を再実験し，新しい診断用 CSV を生成する。
3. Phase 1 の結果から Seed 間ばらつきが大きい Ratio 点を特定し，該当点のみ Phase 2（Seed 10 への増加）を実行する。
4. push_ratio を IID / Non-IID・手法別に比較する図を作成し，「Non-IID では攻撃が紛れやすい」という仮説を検証する。
5. Krum 選択結果の推移と Malicious Ratio 0.0 での精度変動を突き合わせ，Krum 自体の不安定性の要因を考察する。
6. 得られた考察を sotsuron_plan.tex の考察章に統合する。

参考文献

[1] Minghong Fang, Xiaoyu Cao, Jinyuan Jia, Neil Zhenqiang Gong, “Local Model Poisoning Attacks to Byzantine-Robust Federated Learning,” Proceedings of the 29th USENIX Security Symposium (USENIX Security 2020), pp. 1605–1622, 2020.